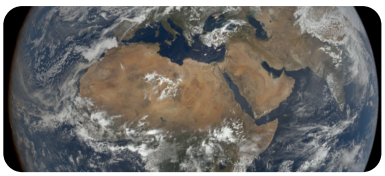
EASY · NEEDS YOUR FREE KEY

EPIC: Earth from space on a date



“A school club wants an ‘Earth from space’ card for any date someone picks.”

1 · Test this URL in your browser first

```
https://api.nasa.gov/EPIC/api/natural/date/2025-06-01?api_key=YOUR_KEY
```

You get a **list** `[ ]` of photos taken that day (≈ 20). Docs: `epic.gsfc.nasa.gov/about/api`

2 · Keep only these fields (per photo)

FIELD	PATH	NOTES
Image name	`photo["image"]`	Not a link yet (see step 3)
Time taken	`photo["date"]`	UTC, text → convert to a date
Centre latitude	`photo["centroid_coordinates"]["lat"]`	Where the camera points
Centre longitude	`photo["centroid_coordinates"]["lon"]`	Cairo ≈ 31.2
Caption	`photo["caption"]`	Optional

3 · Build the picture link from the date + image name

```
https://epic.gsfc.nasa.gov/archive/natural/2025/06/01/jpg/epic_1b_20250601094815.jpg
```

4 · The feature

An “Earth on 1 June 2025” card: the photo centred nearest to Cairo, the time it was taken, and where it points. Notebook: **Part 2 · Card A**.

Trap: a date with no photos returns **200 OK with an empty list** `[ ]`. No error, nothing to show. Check `if not photos:` and show a friendly message.

Done when

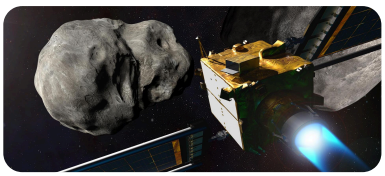
- [ ] Browser URL returns 200 OK
- [ ] Code call returns 200 OK (your key)
- [ ] Clean table: time, lat, lon, image
- [ ] Saved: `epic_clean.csv` downloaded
- [ ] Earth card on screen
- [ ] Empty-date case handled with a message

Stretch (fast pairs)

- Show all of the day’s photos in a row: Earth turning in 24 hours.
- Join a second source by **date + place**: NASA POWER max temperature for the point the camera faces (Card C).

MEDIUM · NEEDS YOUR FREE KEY

NeoWs: closest asteroids this week



“A science club wants a weekly post: which asteroids passed closest, how big, how fast?”

1 · Test this URL in your browser first

```
https://api.nasa.gov/neo/rest/v1/feed?start_date=2025-10-01&end_date=2025-10-07&api_key=YOUR_KEY
```

You get `near_earth_objects`: one **list per date**. Merge the lists into one table. Docs: `api.nasa.gov` → Asteroids - NeoWs

2 · Keep only these fields (per asteroid `a`)

FIELD	PATH	UNIT
Name	`a["name"]`	–
Size	`a["estimated_diameter"]["meters"]["estimated_diameter_min" / "_max"]`	m (use the average)
Speed	`a["close_approach_data"][0]["relative_velocity"]["kilometers_per_second"]`	km/s, **text**
Miss distance	`a["close_approach_data"][0]["miss_distance"]["kilometers"]`	km, **text**
Hazardous?	`a["is_potentially_hazardous_asteroid"]`	true / false

3 · Your twist (pick one or more)

- Another week (max 7 days!) · only the hazardous ones · the biggest instead of the closest · asteroids per day
- Make distances human: `miss_distance_km / 384400` = “times the distance to the Moon”

Notebook: **Part 1** (code-along) and **Part 2 · Card B**.

Traps: more than 7 days → **400**. Speeds and distances arrive as **text** (`"9" > "10"` is true!) → convert with `pd.to_numeric`. NeoWs copies **your key** into every `links` URL, so remove it before you save or share the JSON.

Done when

- [ ] Browser URL returns 200 OK
- [ ] Code call returns 200 OK (your key)
- [ ] Clean table, numbers as numbers
- [ ] Saved: `neows_my_week.csv` downloaded
- [ ] Your list printed on screen
- [ ] Wrong key handled with a clear message

Stretch (fast pairs): join a second source by date

```
https://earthquake.usgs.gov/fdsnws/event/1/query?format=geojson&starttime=2025-10-01&endtime=2025-10-08&minmagnitude=5
```

No key needed. Time is in **milliseconds** since 1970 (UTC); coordinates are **[lon, lat, depth]**. Notebook: **Part 3**.

HARDER · NO KEY NEEDED

NASA POWER: chance of a very hot day



“We’re planning an outdoor event in Cairo. How likely is a day above 35 °C on that date?”

1 · Test this URL in your browser first

```
https://power.larc.nasa.gov/api/temporal/daily/point?parameters=T2M_MAX&community=RE&latitude=30.0444&longitude=31.2357&start=20050101&end=20241231&format=JSON
```

20 years of daily maximum temperature for Cairo (≈ 7,300 days). Dates are `YYYYMMDD`. Docs: `power.larc.nasa.gov/docs`

2 · Keep only these fields

FIELD	PATH	NOTES
Daily values	`data["properties"]["parameter"]["T2M_MAX"]`	A dictionary: `{"20050101": 24.9, …}`
Units	`data["parameters"]["T2M_MAX"]["units"]`	“C”: put it in your column name
Missing marker	`data["header"]["fill_value"]`	-999

3 · Clean, then compute

- Turn the dictionary into a table: one row per date, convert `"20050101"` to a real date.
- Replace **-999** with “missing” and drop those rows. Count them first.
- For a calendar day (e.g. 15 June), take every year ± 3 days → about 140 days. Chance = share of days above 35 °C.
- Save a CSV whose first lines say the **units** and the **source link**.

Notebook: **Part 2 · Card C**. Our result: 15 June in Cairo → **78%** chance of a day above 35 °C.

Trap: missing days are **-999**, not empty. Average them in and Cairo “gets colder”. The most recent days are always -999 (data arrives with a delay): ask for an end date a week back, or clean them.

Done when

- [ ] Browser URL returns 200 OK
- [ ] Code call returns 200 OK
- [ ] -999 rows counted and removed
- [ ] Clean table with units in the name
- [ ] Saved: `city_tmax_daily.csv` downloaded
- [ ] A “% chance” answer on screen

Stretch (fast pairs)

- Add rain and wind in one call: `parameters=T2M_MAX,PRECTOTCORR,WS10M` (°C, mm/day, m/s) → chance of a wet or windy day.
- Try another city: change `latitude` and `longitude` only.

API cheat sheet

Need → find API → read docs → **test URL in browser** → code the call → inspect JSON → pick fields → clean → **save a backup** → build the feature → handle errors

Status codes

`200`	It worked. Still check the list isn't empty `[ ]`
`400`	A parameter is wrong (date format, range too long)
`401/404`	Wrong address. api.nasa.gov answers 401 for a wrong endpoint
`403`	Key wrong or missing (all 40 characters?)
`429`	Too many calls. DEMO_KEY = 10/hour per Wi-Fi; your key = 10,000
`5xx`	Their problem: retry later, use your saved copy

Reading JSON

- `{ }` dictionary → look up by **name**: `d["name"]`
- `[ ]` list → look up by **position**: `lst[0]`
- Lost? `print(list(d.keys()))` and `len(lst)`
- Browser: Firefox shows JSON as a tree. Collapse all, open one box at a time, write the path down.

Clean before you build

- Text → number: `pd.to_numeric(col)`
- -999 → missing: `col.replace(-999, float("nan"))`
- Dates: `pd.to_datetime(col)`; times are usually UTC
- Units in column names: `tmax_c`, `miss_km`
- GeoJSON order is **[lon, lat]**; USGS time is **ms**

The fallback pattern: Python

```
def fetch_or_backup(url, params, backup_path):
    try:
        r = requests.get(url, params=params, timeout=30)
        r.raise_for_status()                          # 4xx/5xx → error
        data = r.json()
        with open(backup_path, "w") as f:             # save the moment it works
            f.write(json.dumps(data).replace(API_KEY, "***"))    # never save your key
        return data
    except Exception as e:
        print("⚠️ Live data unavailable:", e)          # a clear message
        with open(backup_path) as f:                  # …then the saved copy
            return json.load(f)
```

The same in JavaScript (fetch)

```
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const data = await res.json();
  localStorage.setItem("backup", JSON.stringify(data));
  show(data);
} catch (e) {
  showMessage("Live data unavailable, showing saved data");
  show(JSON.parse(localStorage.getItem("backup")));
}
```

Keys

- Get your own: **api.nasa.gov** (1 minute)
- Free NASA key in a web page: OK (worst case: quota used up)
- **Paid key (AI keys!): never** in a web page or public repo
- Colab: 🔑 Secrets panel, not a code cell

AI assistants

- Great for: explaining docs, drafting parsing code
- They invent endpoints and field names: check every field exists in the **real** response
- Never paste a paid key into a prompt or shared code